

Trường ĐH Công Thương TP. HCM Khoa Công nghệ Thông tin Bộ môn Kỹ thuật phần mềm Công nghệ .NET	BÀI 9. LẬP TRÌNH VỚI CƠ SỞ DỮ LIỆU	
--	---	--

A. MỤC TIÊU

Sau khi hoàn thành bài thực hành này, sinh viên có thể:

- Hiểu được ORM và Entity Framework (EF6) và
- Hiểu và vận dụng Mô hình MVVM trong WPF.
- Tạo Entity từ Database (Database First).
- Cấu hình DbContext và Connection String.
- Thực hiện Binding dữ liệu với DataGrid
- Xây dựng ứng dụng CRUD đơn giản với WPF và EF.

B. DỤNG CỤ - THIẾT BỊ THỰC HÀNH CHO MỘT SINH VIÊN

STT	Thiết bị - Phần mềm – Thư viện	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	
2	Microsoft Visual Studio 2019 trở lên (.NET Framework từ 4.7.2)	1	1	
3	Entity Framework (6.x.x)	1	1	
4	Microsoft SQL Server từ 2017	1	1	

C. NỘI DUNG THỰC HÀNH

1. Tóm tắt lý thuyết

Chương 4. Lập trình với cơ sở dữ liệu

4.1. Giới thiệu ORM và Entity Framework

4.1.1. Tổng quan ORM

ORM (Object–Relational Mapping) là kỹ thuật ánh xạ giữa mô hình hướng đối tượng và cơ sở dữ liệu quan hệ, giúp lập trình viên thao tác dữ liệu thông qua các đối tượng thay vì viết câu lệnh SQL trực tiếp.

Entity Framework (EF) là ORM phổ biến trong .NET, hỗ trợ tự động kết nối CSDL, thực hiện CRUD (Create, Read, Update, và Delete), ánh xạ quan hệ giữa các bảng và quản lý dữ liệu một cách an toàn, hiệu quả.

Ví dụ:

```
//Không dùng ORM  
SELECT* FROM SinhVien WHERE MaSV = 'SV01'  
//Dùng EF  
var sv = db.SinhViens.FirstOrDefault(x => x.MaSV == "SV01");
```

4.1.2. Vai trò của ORM

ORM giúp đơn giản hóa và tối ưu hóa việc thao tác dữ liệu trong ứng dụng phần mềm:

- Đơn giản hóa việc truy vấn và thao tác dữ liệu bằng LINQ.
- Giảm lỗi cú pháp SQL và hạn chế SQL Injection.
- Tăng tốc độ phát triển và dễ bảo trì, mở rộng ứng dụng.
- Quản lý quan hệ dữ liệu theo hướng đối tượng.
- Hỗ trợ tracking, transaction và đảm bảo tính nhất quán dữ liệu.

4.1.3. Cài đặt và cấu hình EF

Cách 1: Trong Visual Studio → Tools → NuGet Package Manager → Package Manager Console, chạy:

```
Install-Package EntityFramework
```

Cách 2: Right click Project → Manage NuGet Packages... → Tab Browse → tìm EntityFramework 6.x.x (by Microsoft) → Install

Thư viện thêm vào:

- EntityFramework.dll
- EntityFramework.SqlServer.dll

4.1.4. Các chiến lược phát triển EF

- Database First: Tạo CSDL trước, sinh model từ database.
- Code First: Tạo class trước, tự sinh database qua Migration.
- Model First: Thiết kế mô hình rồi sinh code và database.

4.2. Kiến trúc và thành phần cốt lõi của EF

4.2.1. Đối tượng DbContext, DbSet<T>

a. DbContext

DbContext đại diện cho phiên làm việc (session) với cơ sở dữ liệu, chịu trách nhiệm kết nối, truy vấn và theo dõi dữ liệu.

Ví dụ:

```
internal class AppDbContext : DbContext {  
    public DbSet<Khoa> Khoas { get; set; }  
}
```

b. DbSet<T>

DbSet<T> là tập các đối tượng cùng loại, tương ứng với một bảng trong cơ sở dữ liệu.

Ví dụ:

- DbSet<Khoa> tương ứng bảng Khoa
- DbSet<Lop> tương ứng bảng Lop

Chức năng của DbSet:

- Lấy dữ liệu: `var ds = context.SinhViens.ToList();`
- Thêm: `context.SinhViens.Add(sv);`
- Xóa: `context.SinhViens.Remove(sv);`
- Sửa: Theo dõi tự động qua DbContext

Tóm lại:

- DbContext: “cầu nối” giữa chương trình và database
- DbSet: “bảng trong database”

4.2.2. Chuỗi kết nối (Connection String) và cấu hình App.config

Connection String cho EF biết cách kết nối tới SQL Server:

```
<connectionStrings>  
  <add name="Entities"  
    connectionString="metadata=res://*/...csdl |  
                      res://*/...ssdl | res://*/...msl;  
    provider=System.Data.SqlClient; provider connection string="  
    Data Source=...; Initial Catalog=...; Integrated Security=True;  
    ...  
  </connectionStrings>
```

Các tham số chính:

Tham số	Ý nghĩa
metadata	Đường dẫn đến file ánh xạ EDMX
Data Source	Tên server SQL
Initial Catalog	Tên database
Integrated Security=True	Dùng Windows Authentication
User ID / Password	Khi dùng SQL Authentication

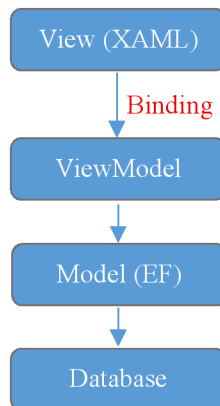
Cách EF sử dụng Connection String:

- Khi tạo DbContext:

`base("QLSVConnection")`

- EF sẽ tìm connection string cùng tên trong App.config khi khởi tạo DbContext.

Trong WPF, EF kết hợp với mô hình MVVM theo sơ đồ:



2. Bài tập thực hành trên lớp

Bài tập có hướng dẫn

Bài 1. Tạo form Quản lý Khoa có giao diện như sau:

Quản lý Khoa

THÔNG TIN KHOA

Mã khoa:

CNTP

Tên khoa:

Công nghệ thực phẩm

Thêm

Sửa

Xóa

DANH SÁCH KHOA

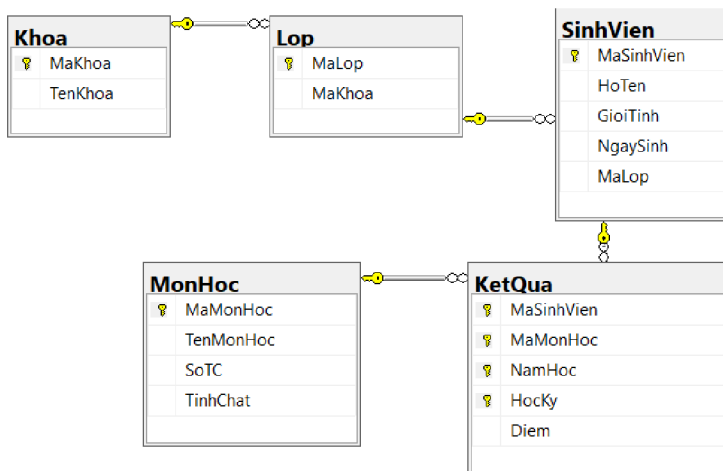
Mã khoa	Tên khoa
CNTP	Công nghệ thực phẩm
CNTT	Công nghệ thông tin
CT	Chính trị
HH	Hóa học
KT	Kế toán

Yêu cầu:

- Kết nối SQL Server.
- Hiển thị danh sách khoa lên DataGrid.
- Binding dữ liệu giữa View ↔ ViewModel ↔ Model.
- Tuân thủ mô hình tách lớp MVVM chuẩn:
 - Model: chứa entity EF
 - ViewModel: xử lý dữ liệu
 - View: XAML giao diện

Hướng dẫn thực hiện:

Bước 1: Chuẩn bị CSDL Quản lý sinh viên gồm các bảng như sau:



Bước 2: Tạo Project đặt tên QLSinhVien_WPF

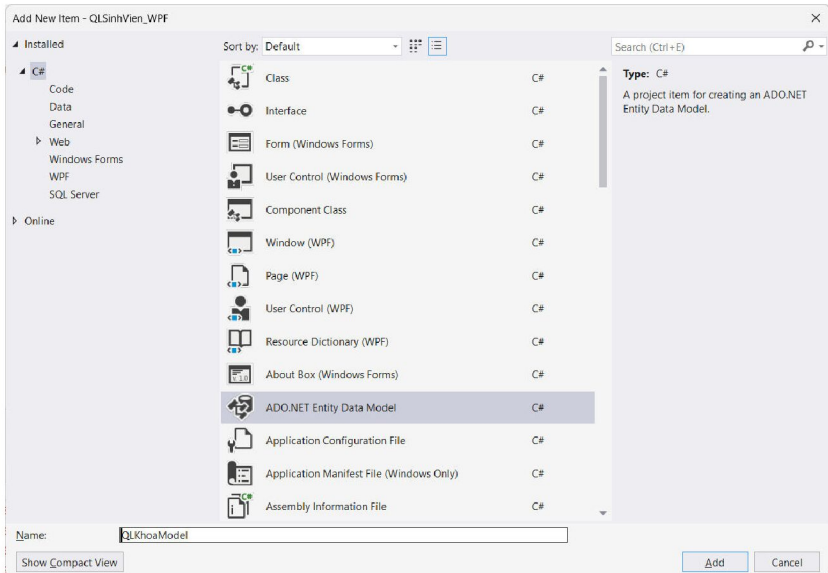
Bước 3: Trong project tạo 3 thư mục:

- Models
- ViewModels
- Views

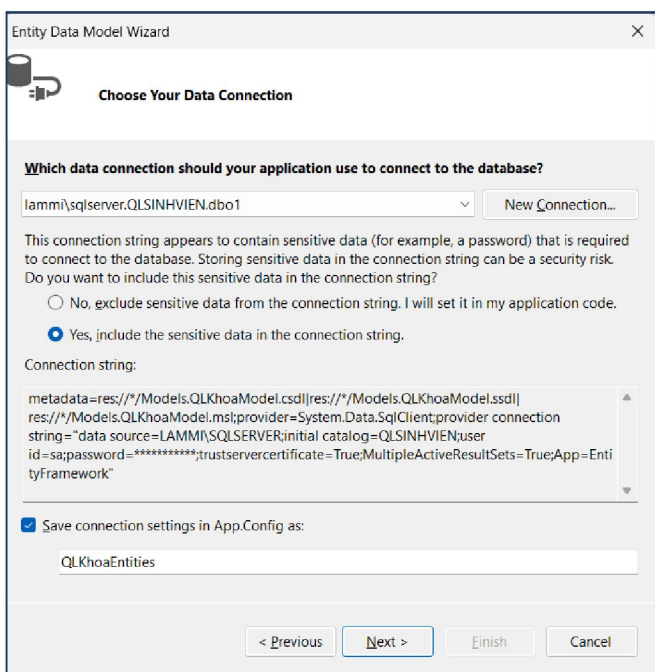
🔗 Lưu ý: Tạo QLKhoaView.xaml trong thư mục Views.

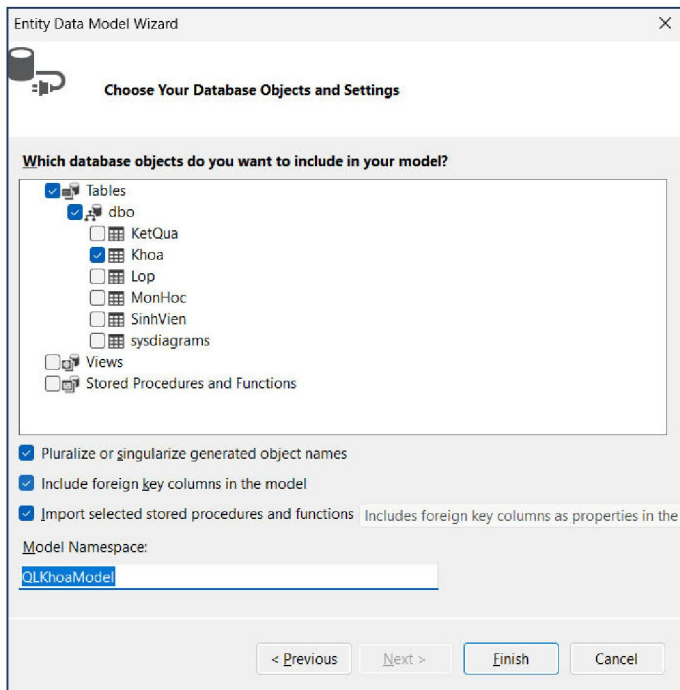
Bước 4: Tạo Entity từ DB (Database First)

Right click folder **Models** → Add → New Item → **ADO.NET Entity Data Model** → Đặt tên: **QLKhoaModel**.

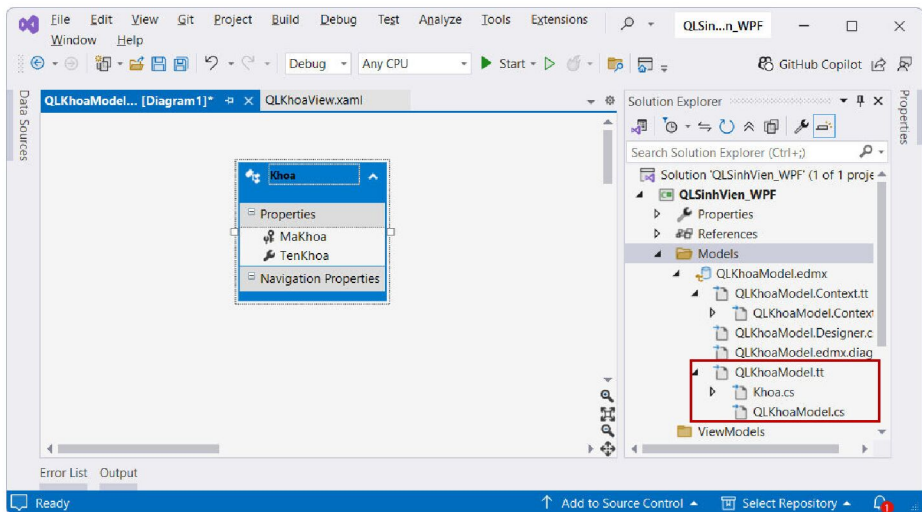


Chọn: **EF Designer from database** → Tạo kết nối SQL Server → Chọn bảng: Khoa





Và kết quả đạt được:



Bước 5: Trong ViewModel tạo các class:

- Class BaseViewModel thực thi interface `INotifyPropertyChanged`.
- Class KhoaViewModel.cs kế thừa class `BaseViewModel`


```

internal class KhoaViewModel: BaseViewModel {
    private QLKhoaEntities db = new QLKhoaEntities();
    public ObservableCollection<Khoa> DS_Khoa { get; set; }
    private Khoa _selectedKhoa;
    public Khoa SelectedKhoa {
        get => _selectedKhoa;
        set {
            _selectedKhoa = value;
            OnPropertyChanged(nameof(SelectedKhoa));
        }
    }
    public KhoaViewModel() {
        LoadData();
    }
    void LoadData() {
        DS_Khoa = new ObservableCollection<Khoa>(db.Khoas.ToList());
        OnPropertyChanged(nameof(DS_Khoa));
    }
}

```

Bước 6:

- Thiết kế giao diện XAML theo hình (trong View)
- Gán dữ liệu lên DataGrid trong WPF (DataGrid Binding)

```

<DataGrid ItemsSource="{Binding DS_Khoa}"
           SelectedItem="{Binding SelectedKhoa}"
           AutoGenerateColumns="False">
    <DataGrid.Columns>
        <DataGridTextColumn Header="Mã khoa"
                           Binding="{Binding MaKhoa}" Width="*" />
        <DataGridTextColumn Header="Tên khoa"
                           Binding="{Binding TenKhoa}" Width="2*" />
    </DataGrid.Columns>
</DataGrid>

```

- Binding dữ liệu vào TextBox

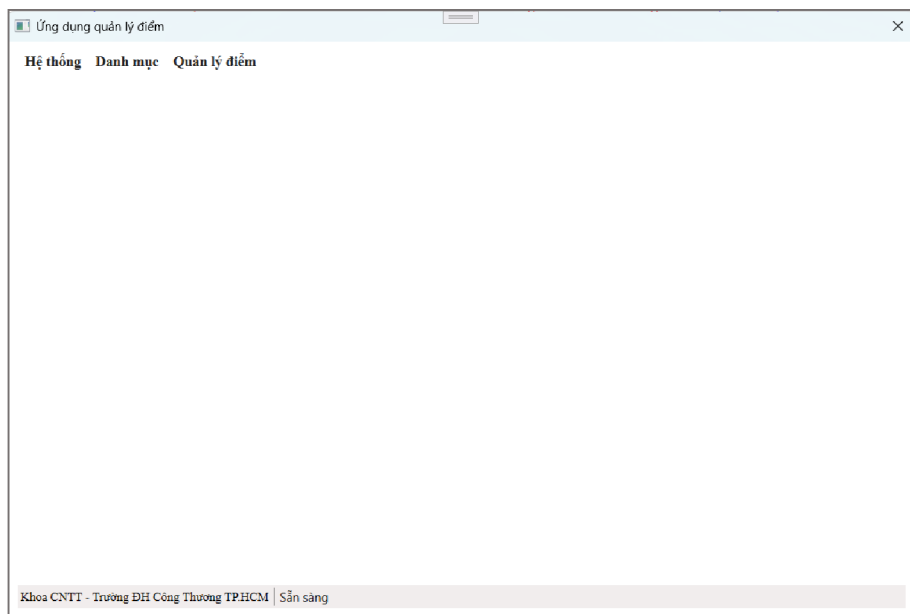
```
<TextBox Text="{Binding SelectedKhoa.MaKhoa}"
        IsEnabled="False"/>
<TextBox Text="{Binding SelectedKhoa.TenKhoa}"/>
```

Bước 7: Thực thi chương trình và kiểm tra kết quả

🔗 Lưu ý: kiểm tra đường dẫn của giao diện QLKhoaView.xaml trong file App.xaml

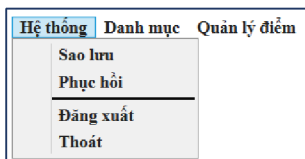
Bài tập sinh viên tự làm

Bài 2. Thiết kế giao diện Form chính – Ứng dụng quản lý điểm:



Yêu cầu:

- Menu chính gồm các mục sau:
 1. Hệ thống: Sao lưu, Phục hồi, Đăng xuất, Thoát.



2. Danh mục: Khoa, Lớp, Môn học, Sinh viên.

Hệ thống	Danh mục	Quản lý điểm
	Khoa Lớp Môn học Sinh viên	

3. Quản lý điểm: Nhập điểm, Xem điểm: Theo Sinh viên, Theo Lớp

Hệ thống	Danh mục	Quản lý điểm
		Nhập điểm Xem điểm ▶
		Theo Sinh viên Theo Lớp

- Khu vực chính (trống) ở trung tâm form dùng để hiển thị UserControl tương ứng với chức năng được chọn (chỉ hiển thị một chức năng tại một thời điểm).
- Thanh trạng thái (StatusStrip).
- Khi người dùng click menu **Danh mục/Khoa**: hiển thị nội dung quản lý Khoa trực tiếp trong vùng nội dung của cửa sổ chính.

Ứng dụng quản lý điểm

Hệ thống Danh mục **Quản lý điểm**

THÔNG TIN KHOA

Mã khoa:

Tên khoa:

DANH SÁCH KHOA

Mã khoa	Tên khoa
CNTP	Công nghệ thực phẩm
CNTT	Công nghệ thông tin
CT	Chính trị
HH	Hóa học
KT	Kế toán

Khoa CNTT - Trường ĐH Công Thương TP.HCM | Đang xem danh sách Khoa

Hướng dẫn:

Bước 1: Chuyển khai báo từ <Window> sang <UserControl>

- Loại bỏ các thuộc tính của Window: Title, WindowStartupLocation, ResizeMode, Icon.
- Giữ nguyên toàn bộ bố cục và các control.

Bước 2: Thay đổi lớp kế thừa từ Window sang UserControl

`public partial class QLKhoaView : UserControl`

(Mọi xử lý sự kiện (Add, Edit, Delete, Save, Clear, ...) vẫn hoạt động bình thường).

Bước 3: Tích hợp vào Form chính khi người dùng click chọn menu Danh mục/ Khoa

Bài 3. Tạo ứng dụng Quản lý lớp có yêu cầu và giao diện như sau:

Yêu cầu:

- Hiện thị danh sách lớp lên DataGrid
 - DataGrid không sử dụng AutoGenerateColumns, các cột phải được khai báo rõ ràng trong XAML.
 - Khi chọn một dòng trong DataGrid:
 - Thông tin Mã lớp hiển thị lên TextBox.
 - Mã khoa tương ứng được chọn đúng trong ComboBox Khoa.
- Binding dữ liệu giữa View ↔ ViewModel ↔ Model.
- Tuân thủ mô hình tách lớp MVVM chuẩn:
 - Model: chứa entity EF (bảng *Lop*, *Khoa*).
 - ViewModel: load danh sách Lớp và danh sách Khoa.

- View: XAML giao diện (DataGrid, TextBox nhập Mã lớp, ComboBox chọn Mã khoa).
- Binding dữ liệu vào ComboBox Khoa:
 - ComboBox hiển thị danh sách Khoa từ ViewModel.
 - DisplayMemberPath: giá trị hiển thị ("TenKhoa")
 - SelectedValuePath: giá trị thực ("MaKhoa")
 - SelectedValue được binding với thuộc tính MaKhoa của Lớp đang được chọn.

Bài 4. Tạo ứng dụng WPF Quản lý sinh viên có yêu cầu và giao diện như sau:

Yêu cầu:

- Hiển thị danh sách sinh viên lên DataGrid.
- Binding dữ liệu giữa View ↔ ViewModel ↔ Model theo mô hình MVVM chuẩn:
 - Model: chứa entity EF (SinhVien, Lop).
 - ViewModel: xử lý dữ liệu, load danh sách lớp, load sinh viên.
 - View: giao diện XAML.
- Binding dữ liệu vào ComboBox Lớp (ItemsSource, SelectedItem / SelectedValue).
- Chọn một dòng trên DataGrid sẽ hiển thị dữ liệu lên các control

TextBox/ComboBox tương ứng.

3. Bài tập về nhà

Bài 5. Tạo ứng dụng WPF Quản lý môn học có yêu cầu và giao diện như sau:

The screenshot shows a WPF application window titled "Quản lý Môn học". Inside the window, there is a form with the following elements:

- Four input fields: "Mã môn:", "Tên môn học:", "Số TC:", and "Tính chất:". The "Tính chất:" field is a dropdown menu currently showing "Tự chọn".
- Five buttons: "Thêm", "Sửa", "Xóa", "Lưu", and "Hủy".
- A DataGrid at the bottom with four columns: "Mã môn", "Tên môn học", "Số TC", and "Tính chất".

Yêu cầu:

- Hiển thị danh sách Môn học lên DataGrid.
- Binding dữ liệu giữa View ↔ ViewModel ↔ Model theo đúng mô hình MVVM.
- Tuân thủ mô hình tách lớp MVVM chuẩn:
 - Model: chứa entity EF từ Database First (MonHoc).
 - ViewModel: xử lý dữ liệu, cung cấp ObservableCollection, SelectedItem.
 - View: giao diện XAML, binding vào ViewModel.
- Binding dữ liệu từ ViewModel vào các TextBox:
 - MaMonHoc
 - TenMonHoc
 - SoTC
 - TinhChat

- Khi chọn một dòng trong DataGrid → dữ liệu phải hiển thị lên các control tương ứng.
- DataGrid không được AutoGenerateColumns, phải định nghĩa rõ các cột.

Bài 6. Tạo ứng dụng WPF Quản lý nhập điểm có yêu cầu và giao diện như sau:

The screenshot shows a WPF application window titled "Quản lý nhập điểm". The window contains three dropdown menus for "Môn học:", "Năm học:", and "Học kỳ:". Below these are two buttons: "Tải danh sách sinh viên" and "Lưu điểm". At the bottom, there is a DataGrid with four columns: "Mã SV", "Họ tên", "Lớp", and "Điểm".

Yêu cầu:

- Hiển thị danh sách sinh viên lên DataGrid theo bộ lọc:
 - Môn học
 - Năm học
 - Học kỳ
- Binding dữ liệu giữa View ↔ ViewModel ↔ Model theo mô hình MVVM.
 - Model: entity EF từ Database First (SinhVien, MonHoc, KetQua).
 - ViewModel: xử lý dữ liệu, tải danh sách sinh viên theo bộ lọc.
 - View: giao diện XAML (ComboBox, Button, DataGrid).
- Cho phép chọn dữ liệu lọc qua 3 ComboBox:
 - ComboBox Môn học: hiển thị danh sách môn học từ bảng MonHoc.
 - ComboBox Năm học: danh sách năm học (tự định nghĩa).

- ComboBox Học kỳ: giá trị 1, 2, 3.
- Nút “Tải danh sách sinh viên”:
 - Kiểm tra đã chọn đủ Môn học – Năm học – Học kỳ.
 - Sau khi hợp lệ → tải danh sách sinh viên và hiển thị lên DataGrid.
- DataGrid hiển thị danh sách sinh viên:
 - Mã sinh viên
 - Họ tên
 - Lớp
 - Điểm (cột cho phép nhập giá trị nhưng chưa yêu cầu lưu)

Bài 7: Tạo ứng dụng WPF hiển thị bảng điểm của sinh viên có yêu cầu và giao diện như sau:

The screenshot shows a WPF application window titled "Xem điểm". The window contains a form with the following elements:

- Title Bar:** "Xem điểm" with standard window controls.
- Section Header:** "BẢNG ĐIỂM SINH VIÊN" in blue text.
- Search Section:**
 - Input field: "Mã sinh viên:" followed by a "Tìm" button.
 - Input field: "Họ tên:"
 - Input field: "Lớp:"
- Filter Section:**
 - Dropdown menu: "Năm học:"
 - Dropdown menu: "Học kỳ:"
 - Button: "Xem bảng điểm"
- DataGrid:**

STT	Mã MH	Tên môn học	Số TC	Điểm	Điểm chữ
-----	-------	-------------	-------	------	----------
- Summary Section:**
 - Label: "Tổng Tin chỉ:"
 - Label: "GPA:"
 - Label: "Xếp loại:"

Yêu cầu:

- Chức năng của form: tra cứu và hiển thị bảng điểm của sinh viên theo từng năm học và từng học kỳ.
- Nhập mã sinh viên vào TextBox và chọn Button Tìm. Nếu tìm thành công, hiển thị thông tin sinh viên vào đúng vị trí trên giao diện.

- Kiểm tra tính hợp lệ:
 - Không cho phép tìm khi mã sinh viên rỗng. Nếu không tìm thấy sinh viên sẽ xuất thông báo cho người dùng.
 - Người dùng phải chọn đầy đủ năm học và học kỳ trước khi nhấn Xem bảng điểm, hệ thống tải dữ liệu bảng điểm tương ứng.
- Hiện thị danh sách các môn học của sinh viên trong học kỳ đã chọn (Bảng chỉ cho phép xem, không chỉnh sửa trực tiếp).
- Thống kê:
 - Tính và hiển thị: Tổng số tín chỉ của học kỳ, Điểm trung bình (GPA) của học kỳ, Xếp loại học lực.
 - GPA được tính tự động từ điểm các môn và số tín chỉ.
 - Xếp loại học lực dựa trên GPA theo quy định.